

SQL – DDL

(Data Definition Language)

Data Definition Language (DDL) is the part of SQL used to define and manage the structure of a database. DDL commands are used to create, modify, and remove database objects such as tables, databases, indexes, and constraints.

While **DML (Data Manipulation Language)** works with the data stored inside tables, **DDL works with the structure of those tables and other database objects.**

Common DDL commands include:

1. **CREATE** – creates a new database object.
2. **ALTER** – changes the structure of an existing database object. *Support and behavior can vary by DBMS.*
3. **DROP** – removes a database object.

1. CREATE

The CREATE command is used to create new database and database objects.

For example:

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Major VARCHAR(50)  
);
```

This statement creates a table named Student with three columns.

The table structure includes:

- column names
- data types
- constraints such as PRIMARY KEY

For **MySQL**, the most common objects are:

Object	Purpose	Example
Database / Schema	Container for tables and other objects	CREATE DATABASE SchoolDB;
Table	Stores the actual data in rows and columns	CREATE TABLE Student (...);
View	A saved query that behaves somewhat like a virtual table	CREATE VIEW CSStudents AS ...;
Index	Helps speed up searching and retrieval	CREATE INDEX idx_name ON Student(Name);
Stored Procedure	A stored set of SQL statements that performs a task	CREATE PROCEDURE GetStudents() ...
Function	A stored routine that returns a value	CREATE FUNCTION CalculateAge(...) ...
Trigger	SQL code that runs automatically when an event occurs	CREATE TRIGGER ... BEFORE INSERT ...
Event	SQL code scheduled to run automatically at a certain time	CREATE EVENT ...

CREATE Command with the DATABASE Object

The CREATE command is used to create database objects. When used with the DATABASE object, it creates a new database.

Example:

```
CREATE DATABASE SchoolDB;
```

This creates a new database named SchoolDB.

In MySQL, creating the database does not automatically make it the active database. You normally follow it with the USE command:

```
CREATE DATABASE SchoolDB;  
USE SchoolDB;
```

After USE SchoolDB;, any tables or other objects you create will be placed inside that database unless another database is specified.

Using IF NOT EXISTS

You can also write:

```
CREATE DATABASE IF NOT EXISTS SchoolDB;
```

IF NOT EXISTS is an **optional clause** that tells MySQL to create the database only if a database with that name does not already exist.

In **MySQL**, the terms **database** and **schema** are essentially used interchangeably, so:

```
CREATE DATABASE SchoolDB;
```

and

```
CREATE SCHEMA SchoolDB;
```

have the same effect.

CREATE Command with the TABLE Object

The CREATE TABLE command is a **DDL (Data Definition Language)** command used to create a new table in a database.

```
CREATE TABLE Student (  
    StudentID INT,  
    Name VARCHAR(50),  
    Major VARCHAR(50) );
```

Optional Constraints

You can also include **optional constraints** and **optional clause** when creating the table:

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Major VARCHAR(50) );
```

Constraints are generally optional when defining a table. You can create a table without any constraints at all.

However, without constraints, the DBMS does not enforce many important rules. For example, it could allow duplicate StudentID values or NULL values.

Common SQL constraints include PRIMARY KEY, NOT NULL, FOREIGN KEY, and UNIQUE.

A PRIMARY KEY constraint automatically enforces both **NOT NULL** and **UNIQUE**.

This means a primary key column:

- cannot contain a NULL value
- cannot contain duplicate values

Different Data Types in SQL

A **data type** specifies what kind of value can be stored in a column. When creating a table, each column is assigned a data type.

For example:

```
CREATE TABLE Product (  
  ProductID INT,  
  ProductName VARCHAR(100),  
  Price DECIMAL(8,2)  
);
```

Here, INT, VARCHAR(100), and DECIMAL(8,2) are data types.
The most common SQL data types fall into several categories:

Category	Common Data Types	Used For
Numeric	INT, SMALLINT, BIGINT, DECIMAL, FLOAT, DOUBLE	Numbers
Character / String	CHAR, VARCHAR, TEXT	Text
Date and Time	DATE, TIME, DATETIME, TIMESTAMP	Dates and times
Boolean	BOOLEAN, BOOL	True/false values

Numeric Data Types

INT stores whole numbers:

Examples:

5

-10

DECIMAL(p,s) stores exact decimal values:

Price DECIMAL(6,2)

Here:

6 = total number of digits

2 = number of digits after the decimal point

Examples:

19.99

1250.50

DECIMAL is commonly used for money because it stores exact decimal values.

FLOAT store approximate values and are often used for scientific or measurement data.

Temperature FLOAT

Example:

3.1415926536

String Data Types

VARCHAR(n) stores variable-length text up to the specified maximum length.

Example:

ProductName VARCHAR(100)

Date Data Types

DATE stores a date:

BirthDate DATE

Example:

2026-09-27

Boolean Data Type

A Boolean represents a true/false condition.

IsAvailable BOOLEAN

Typical values are:

TRUE

FALSE

Optional Clause

An **optional clause** is a part of an SQL statement that is **not required for the statement to be valid**, but can be added to change or refine its behavior.

For example:

```
CREATE DATABASE IF NOT EXISTS SchoolDB;
```

Here, IF NOT EXISTS is optional. It tells MySQL to create the database only if it does not already exist.

Another example:

```
DROP DATABASE IF EXISTS SchoolDB;
```

This prevents an error if the database does not exist.

Optional clauses provide additional instructions or conditions but are not always required.

Examples of Optional Clauses

Optional Clause	Example	Purpose
IF NOT EXISTS	CREATE TABLE IF NOT EXISTS Student (...)	Creates the object only if it does not already exist
IF EXISTS	DROP TABLE IF EXISTS Student;	Drops the object only if it exists
WHERE	SELECT * FROM Student WHERE Major = 'Biology';	Filters rows
ORDER BY	SELECT * FROM Student ORDER BY Name;	Sorts the result
GROUP BY	SELECT Major, COUNT(*) FROM Student GROUP BY Major;	Groups rows together
HAVING	... GROUP BY Major HAVING COUNT(*) > 5;	Filters groups
LIMIT	SELECT * FROM Student LIMIT 10;	Limits the number of rows returned
DISTINCT	SELECT DISTINCT Major FROM Student;	Removes duplicate results

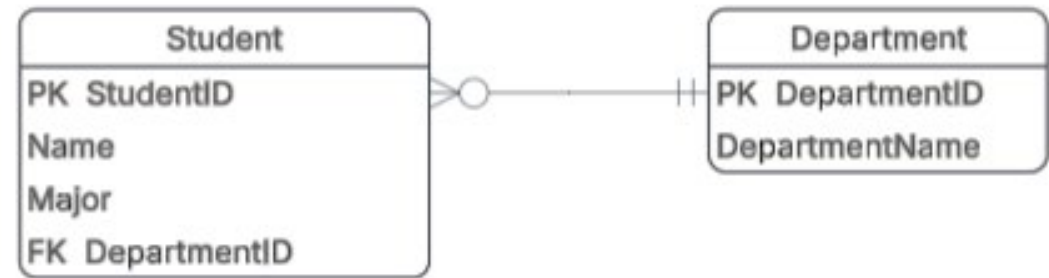
Creating a Table with a Foreign Key

Tables can also be related to one another using a foreign key.

```
CREATE TABLE Department (  
    DepartmentID INT PRIMARY KEY,  
    Department Name VARCHAR(50)  
);
```

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Major VARCHAR(50),  
    DepartmentID INT NOT NULL,  
    FOREIGN KEY (DepartmentID)  
    REFERENCES Department(DepartmentID)  
);
```

Here, DepartmentID in the Student table is a **foreign key** that refers to DepartmentID in the Department table.



2. ALTER

The ALTER command is used to modify the structure of an existing table.

For example:

```
ALTER TABLE Student  
ADD Email VARCHAR(100);
```

This adds a new column named Email to the Student table.

Depending on the DBMS, ALTER TABLE can also be used to modify columns, add constraints, or remove columns. You can add multiple columns, each clause separated by a comma.

ALTER TABLE modifies the structure of an existing table. It usually preserves the existing data, but certain structural changes may affect or be restricted by the data already stored in the table.

For example, changing a column's data type, reducing its size, or adding a NOT NULL or UNIQUE constraint can conflict with values already stored.

You can also remove a column:

```
ALTER TABLE Student  
DROP COLUMN Email;
```

This would remove the column email from the Student table.

To change the definition of a column in MySQL, you can use MODIFY:

```
ALTER TABLE Student  
MODIFY COLUMN Name VARCHAR(100);
```

This would change the max length of Name to 100.

This does not work in SQLite.

You can also rename a column:

```
ALTER TABLE Student  
RENAME COLUMN Name TO StudentName;
```

SQL syntax and capabilities differ among DBMSs:

MODIFY COLUMN and ADD CONSTRAINT is available in MySQL, but not in SQLite.

Another common use of ALTER TABLE is adding a constraint after the table has already been created.

For example:

```
ALTER TABLE Student  
  ADD CONSTRAINT uq_email UNIQUE (Email);
```

This also does not work in SQLite.

You can also perform different types of alterations in the same statement:

```
ALTER TABLE Student  
  MODIFY Name VARCHAR(150),  
  ADD Email VARCHAR(100),  
  DROP COLUMN Major;
```

An important point is that ALTER changes the **structure of the table**, *not necessarily the data already stored in it.*

3. DROP

The DROP command removes an entire database or database object.

For example:

```
DROP TABLE Student;
```

This removes both the Student table structure and all data stored in it.

A conditional clause can also be used:

```
DROP TABLE IF EXISTS Student;
```

IF EXISTS prevents an error if the table does not exist.

In MySQL, you can also remove an entire database:

```
DROP DATABASE IF EXISTS SchoolDB;
```

Using **DROP** with Caution:

The **DROP** command should be used with caution, especially in a real-world business environment. This is because it **permanently removes database objects, such as tables or databases, along with the data stored in them.**

While it is appropriate to use **DROP** in a classroom or learning environment where databases can easily be recreated, it is used much more carefully in production systems because accidental data loss can have serious consequences.

In a Real-World Business Setting

DDL statements are typically used by database administrators, developers, or system designers when creating or changing the database structure.

For example, if a company decides that every customer record should include an email address, a developer may use an ALTER TABLE statement to add an Email column.

DDL therefore defines the framework in which the organization's data is stored.